

Route路由設定

- **5-1: Route路由規則**

Route路由設定.....	5-2
自訂規則與Route參數.....	5-4
設定Controller層級的屬性路由.....	5-6
設定Action層級的屬性路由.....	5-7

- **5-2: Attribute Routing**

HTTP Method Attributes.....	5-10
Attribute Routing條件約束.....	5-11

Module 05

Route 路由設定 -1

- 在 ASP.NET Core MVC 中，網址路徑（URL）與 Controller / Action 的對應關係是透過 Routing（路由）來定義的。
- 路由不再設定於 App_Start/RouteConfig.cs，而是集中在 Program.cs 中進行設定，常見方式包含：
 - Convention-based Routing（慣例路由）：使用 MapControllerRoute(...) 設定預設路由規則。
 - Attribute Routing（屬性路由）：在 Controller / Action 上使用 [Route]、[HttpGet] 等標籤定義路由程式碼結構更簡潔明確。

Route路由設定 -2

- 慣例路由是透過 **Program.cs** 預先定義一組通用的網址規則，讓系統依照既定慣例，自動對應到 **Controller** 與 **Action**。

```
app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

- 路由規則說明：
 - {controller}：對應 **Controller** 類別名稱（去除 **Controller**）
 - {action}：對應 **Action** 方法名稱
 - {id?}：可選參數（？表示可有可無）=Home / =Index：預設值

自訂規則與Route參數 -1

- **Program.cs :**

```
app.MapControllerRoute(  
    name: "sayHello",  
    pattern: "Hello/{firstName}/{lastName?}",  
    defaults: new { controller = "About", action = "SayHello" });
```

- **Action :**

```
public IActionResult SayHello(string firstName, string lastName)  
{  
    ViewBag.Message = $"Hello , {firstName} {lastName}";  
    return View("Index");  
}
```

自訂規則與Route參數 -2

- 在設定自訂 **Route** 規則時，建議將條件最明確、比對最嚴格的路由放在最上方，再依序排列到條件較為寬鬆的路由規則。由於路由是由上而下依序比對，如此可避免過於寬鬆的規則搶先命中，才能更精確地對應正確的 **URL** 路徑。

```
app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "sayHello",
    pattern: "Hello/{firstName}/{lastName?}",
    defaults: new { controller = "About", action = "SayHello" });

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

設定Controller層級的屬性路由

- [Route (“自訂名稱”)]：於Controller上方設定。
- 可設定預設執行的Controller/Action，可與Action層級的[Route]設定擇一使用，{ }內可設定參數。

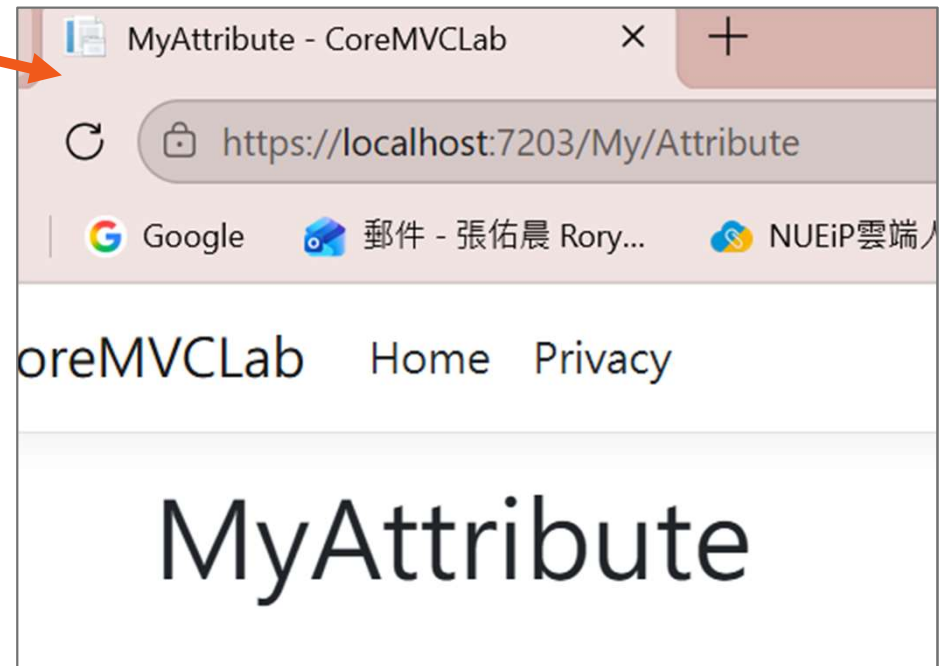
```
[Route("myRoute/[controller]/[action]")]
0 個參考
public class MapRouteController : Controller
{
    [HttpGet("{id:int}")]
    0 個參考
    public IActionResult Index(int id)
    {
        ViewBag.Message = $"你傳入的id值為: {id}";
        return View();
    }
}
```



設定Action層級的屬性路由 -1

- [Route(“自訂名稱”)]：於Action上方設定，與Controller層級的[Route]設定擇一使用。

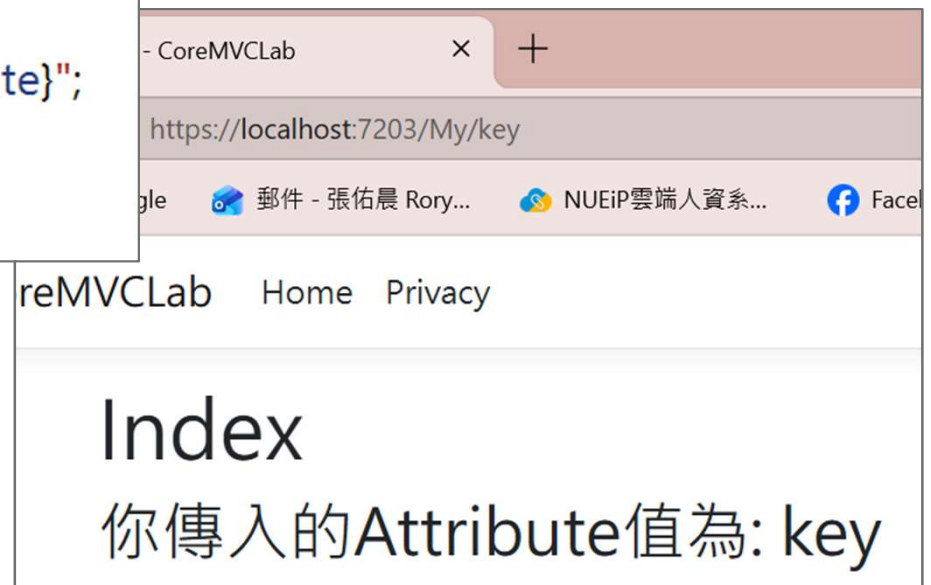
```
[Route("My/Attribute")]  
0 個參考  
public IActionResult MyAttribute()  
{  
    return View();  
}
```



設定Action層級的屬性路由 -2

- [Route(“自訂名稱/{參數}”)]：可設定參數。

```
[Route("My/{Attribute}")]
0 個參考
public IActionResult MyAttributeParam(string Attribute)
{
    ViewBag.Message = $"你傳入的Attribute值為: {Attribute}";
    return View("Index");
}
```



設定Action層級的屬性路由 -3

- **[Route(“自訂名稱/{參數:條件約束}”)]**：設定路由約束Route Constraints，當路由符合設定的條件約束時才進入Action。

```
[Route("myActionRoute/{ShowID:int}")]
```

0 個參考

```
public ActionResult test1(int? ShowID)
```

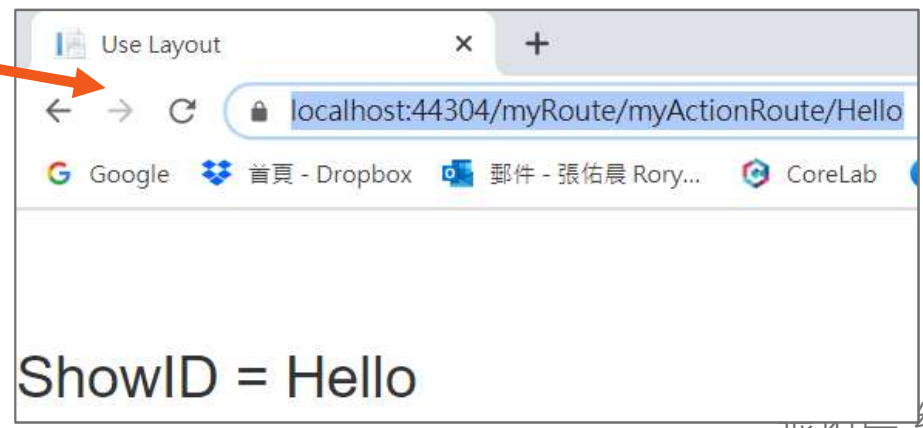
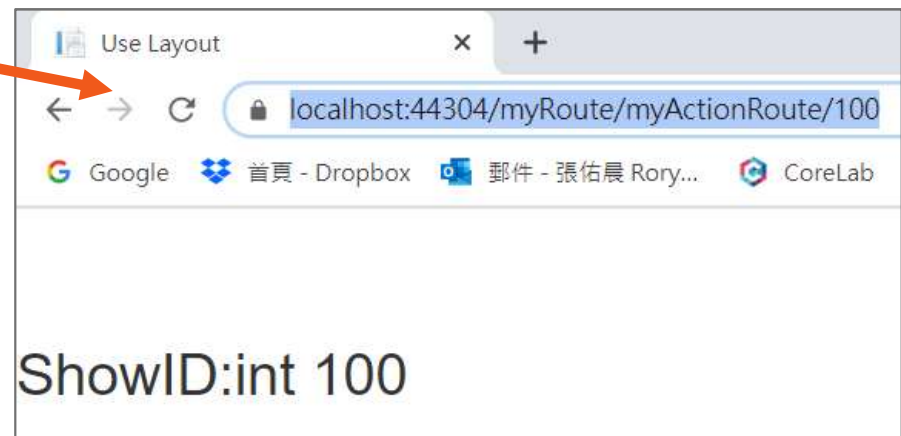
```
{  
    ViewBag.ShowID = ShowID;  
    return View();  
}
```

```
[Route("myActionRoute/{ShowID}")]
```

0 個參考

```
public ActionResult test2(string ShowID)
```

```
{  
    ViewBag.ShowID = ShowID;  
    return View();  
}
```



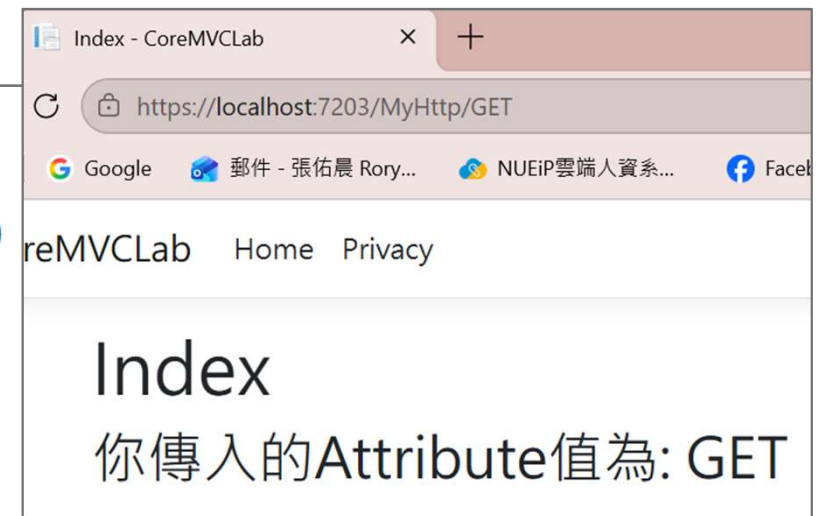
HTTP Method Attributes

- 這些 Attribute 用來限制 Action 可接受的 HTTP 方法，並可搭配路由參數。
 - [HttpGet]：通常用於 取得資料 (Read)，不應改變伺服器端狀態。
 - [HttpPost]：新增 / 提交資料 (Create)
 - [HttpPut]：完整更新資料 (Replace / Update)
 - [HttpDelete]：刪除資料 (Delete)

```
[HttpGet("MyHttp/{Attribute}")]
```

0 個參考

```
public IActionResult MyHttpMethodAttribute(string Attribute)  
{  
    ViewBag.Message = $"你傳入的Attribute值為: {Attribute}";  
    return View("Index");  
}
```



Attribute Routing 條件約束

- [Route]可透過條件約束限制參數的資料型態。

條件約束	描述	範例
alpha	比對大寫或小寫拉丁字母字元 (a-z , A-Z)	{xalpha}
bool	比對布爾值。	{xbool}
Datetime	比對 DateTime 值。	{xdatetime}
decimal	比對十進位值。	{xdecimal}
double	比對 64 位元浮點值。	{xdouble}
float	比對 32 位元浮點值。	{xfloat}
guid	比對 GUID 值。	{xguid}
int	比對 32 位元整數值。	{xint}
length	比對具有指定長度的字串，或在指定的長度範圍內。	{xlength(6)} {xlength(1,20)}
long	比對 64 位元整數值。	{xlong}
max	比對具有最大值的整數。	{xmax(10)}
maxlength	比對長度上限的字串。	{xmaxlength(10)}
分鐘	比對具有最小值的整數。	{xmin(10)}
minlength	比對長度下限的字串。	{xminlength(10)}
range	比對值範圍內的整數。	{xrange(10,50)}
Regex	比對規則運算式。	{xregex(^d{3}-d{3}-d{4}\$)}

參考資料：<https://docs.microsoft.com/zh-tw/aspnet/web-api/overview/web-api-routing-and-actions/attribute-routing-in-web-api-2>